

Helm Charts Tutorial (Beginner) — create, deploy (OpenShift), and store

This is the **hands-on tutorial** part.

If you want the “what is Helm / structure / workflow” overview, read:

- [dataengineering/cloud/helm-charts-intro.md](#)
-

1) Goal (what we will build)

We will:

- create a chart
 - configure it to deploy **nginx**
 - render the YAML (“compile”)
 - deploy it to an OpenShift project
 - learn 3 simple ways to store/share the chart
-

2) Requirements

- Helm installed: `helm version`
 - OpenShift CLI installed: `oc version`
 - Access to a cluster (login credentials)
-

3) Create a starter chart

```
``bash mkdir -p helm-demo && cd helm-demo helm create hello-chart ``
```

Chart structure reminder:

Helm chart structure (what's inside a chart folder)

```
mychart/
  Chart.yaml      (name/version)
  values.yaml     (your settings)
  templates/      (YAML templates)
    deployment.yaml
    service.yaml
    route.yaml    (OpenShift)
  charts/         (optional deps)
```

4) Configure the app to deploy (values)

Edit `hello-chart/values.yaml` and set a real image. Example:

```
```yaml
replicaCount: 1

image: repository: nginx tag: "1.25"

service: type: ClusterIP port: 80 ```
```

---

### 5) Validate the chart

```
```bash
helm lint ./hello-chart ```
```

6) Render (“compile”) to YAML locally

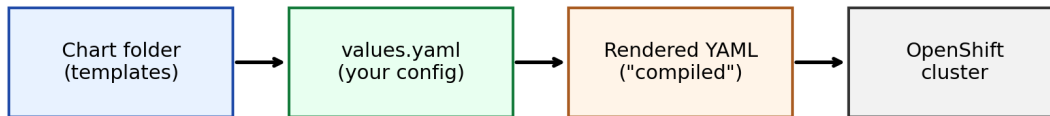
This is the safest step before you touch the cluster:

```
```bash
helm template hello ./hello-chart ```
```

If the output YAML looks wrong, fix `values.yaml` or `templates` and rerun.

Workflow reminder:

Helm workflow: chart → rendered YAML → cluster



Helm does two jobs: 1) render templates to final YAML 2) apply that YAML to the cluster + store a release record

---

## 7) Deploy to OpenShift

### Step A — Login and choose a project

```
``bash oc login https://api.:6443 oc new-project hello-project
```

**or**

```
oc project hello-project ``
```

### Step B — Install the chart into that namespace

```
``bash helm install hello ./hello-chart -n hello-project ``
```

Verify:

```
``bash helm list -n hello-project oc get all -n hello-project ``
```

### Step C — Upgrade (after changing values/templates)

```
``bash helm upgrade hello ./hello-chart -n hello-project ``
```

### Step D — Uninstall

```
``bash helm uninstall hello -n hello-project ``
```

---

## 8) OpenShift routing: Ingress vs Route (beginner view)

- Kubernetes often uses **Ingress**.
- OpenShift commonly uses **Route**.

If you want a URL on OpenShift, you may add a Route template.

Example hello-chart/templates/route.yaml:

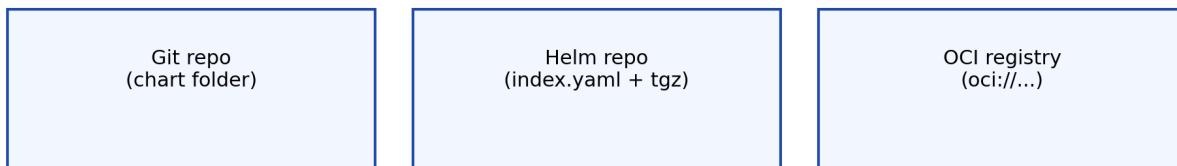
```
``yaml apiVersion: route.openshift.io/v1 kind: Route metadata: name: {{
include "hello-chart.fullname" . }} spec: to: kind: Service name: {{
include "hello-chart.fullname" . }} port: targetPort: http ``
```

Tip: teams usually make this optional with a value like `route.enabled: true`.

---

## 9) How to store/share your chart (3 easy options)

How teams store/share Helm charts (3 common ways)



CI/CD typically runs: `helm upgrade --install ...` (from Git, repo, or OCI)

### Option 1 — Store chart folder in Git (simplest)

```
``bash helm upgrade --install hello ./hello-chart -n hello-project -f
values-prod.yaml ``
```

### Option 2 — Helm repo (package + index)

```
``bash helm package ./hello-chart helm repo index . ``
```

Host the `.tgz` + `index.yaml` (GitHub Pages, S3, Nexus/Artifactory).

### Option 3 — OCI registry (modern)

High-level flow:

```
```bash helm package ./hello-chart helm registry login helm push hello-chart-0.1.0.tgz oci:///charts
helm install hello oci:///charts/hello-chart -n hello-project ```
```

10) Final mental model

- Chart = blueprint
 - Values = settings
 - Render (“compile”) = blueprint → final YAML
 - Install/upgrade = apply YAML to OpenShift
 - Store/share = Git or Helm repo or OCI registry
-

11) Your case: JFrog (Artifactory) + OpenShift — every call, step by step

The most common misunderstanding:

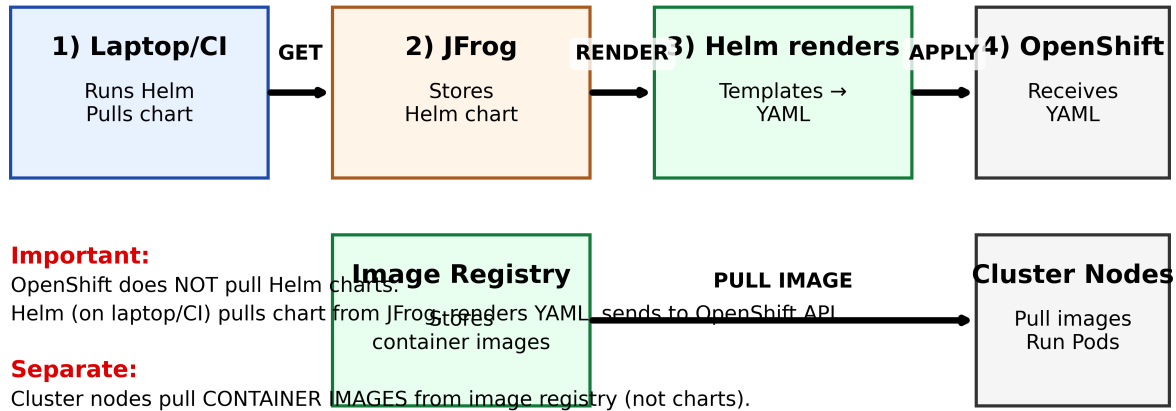
The OpenShift cluster usually does NOT pull the Helm chart.

What really happens:

- **Your laptop / CI runner** pulls the chart from **JFrog**.
- Helm renders templates into final YAML (“compile”).
- Helm sends that YAML to the **OpenShift API**.
- The cluster creates Pods, and the **nodes pull container images** from an image registry (not the chart).

Diagram: who talks to who

JFrog + Helm + OpenShift Flow



The calls (in order)

0) Login to OpenShift

```
``bash oc login https://api.<cluster>:6443 oc project hello-project ``
```

1) Get the chart from JFrog (two common ways)

A) If JFrog is a Helm repo

```
``bash helm repo add myjfrog
https://<jfrog>/artifactory/api/helm/<helm-repo> helm repo update helm
search repo myjfrog ``
```

Then install by name:

```
``bash helm upgrade --install hello myjfrog/hello-chart -n hello-project
-f values-prod.yaml ``
```

B) If JFrog is an OCI registry

```
``bash helm registry login <jfrog> helm pull
oci://<jfrog>/<oci-repo>/hello-chart --version 0.1.0 ``
```

Then install from OCI:

```
``bash helm upgrade --install hello oci://<jfrog>/<oci-repo>/hello-chart
-n hello-project -f values-prod.yaml ``
```

2) Render (“compile”) happens locally

Helm renders templates locally (CI/laptop):

```
``bash helm template hello ./hello-chart ``
```

(If installing from a repo/OCI, Helm does the same rendering step internally before applying.)

3) Apply YAML to the OpenShift API

This happens when you run:

```
```bash helm install hello ... -n hello-project
```

**or**

```
helm upgrade --install hello ... -n hello-project ```
```

### 4) Cluster pulls container images

Then the cluster creates Pods. The nodes pull the **container image** (e.g., `nginx:1.25` or your app image) from your image registry.